Report: Snort IDS Practice

4th Year ITM Information Security Assignment

Client: 20102105 Kim Seungjun

Server: 22101997 Park Seonghun

Snort Logging with Using NMAP

Loading Container

```
# chmod u+x ./start.sh
./start.sh
```

chmod u+x grants execute permission on start.sh for the file owner. ./start.sh then runs the script, which starts the Docker container for this lab environment.

Running Snort

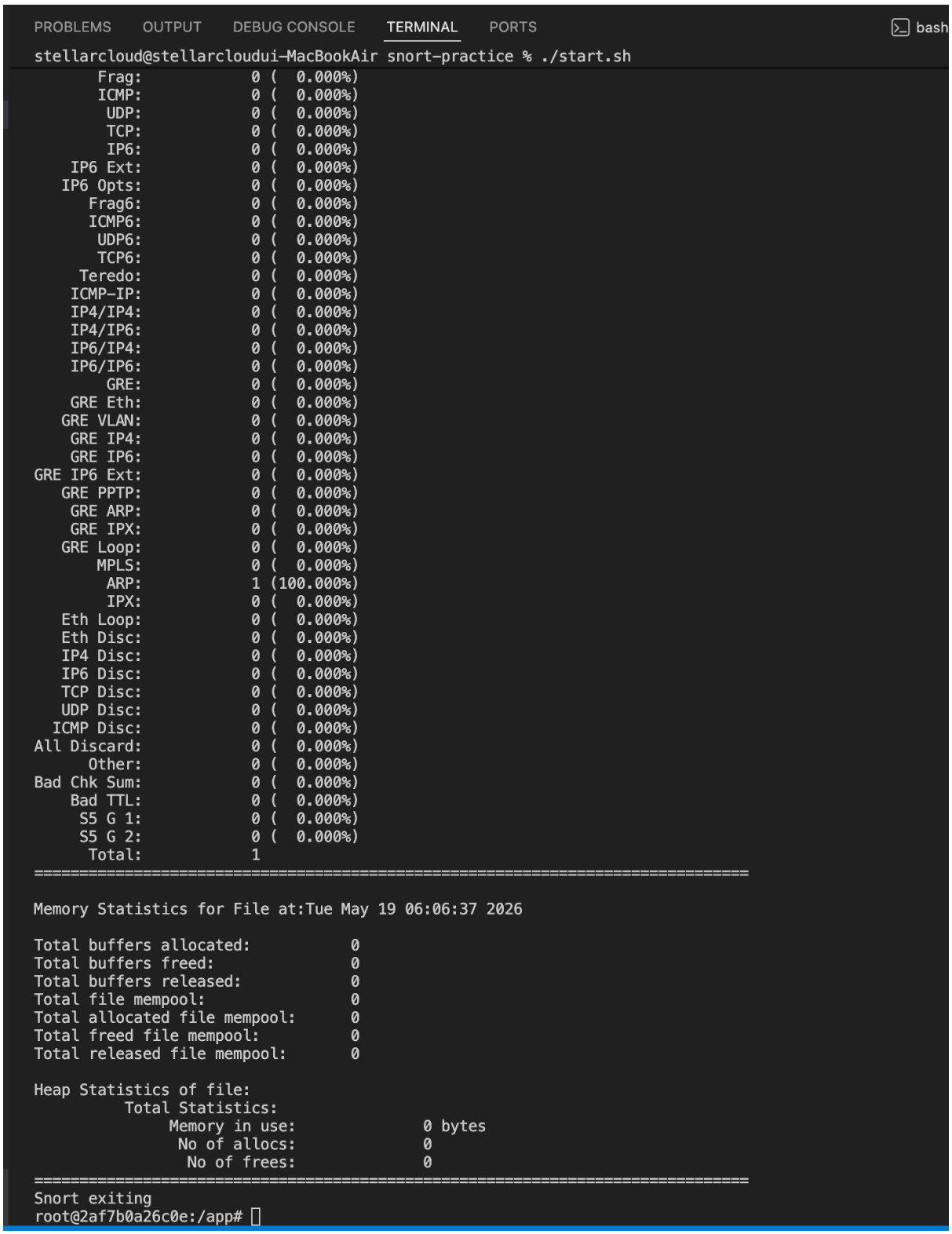
```
# Capturing 1 packet that is Raw Packet(Binary) and Exited
snort -b -n 1
```

-b tells Snort to write captured packets in raw binary (tcpdump-compatible) format to a log file. -n 1 limits the session to exactly one packet before Snort exits automatically.

Send curl request

```
# Just for checking the installation Snort
curl localhost:1234
```

Capturing Result



1. The host sends a request to localhost:1234. (Assuming port forwarding is configured)

2. The host's network stack forwards this packet to Docker's virtual network bridge

interface (e.g., docker0).

3. The Docker engine first broadcasts an ARP (Address Resolution Protocol) request over the bridge network to find the destination container's IP.

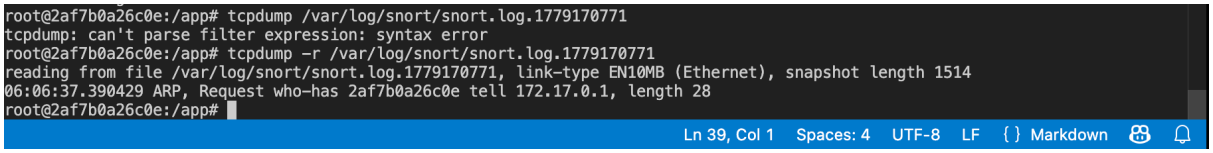
4. Snort inside the container collects this ARP packet (1 packet).

5. However, the TCP packets that should have followed are not caught by Snort at all, and the process terminates with Total: 1 (ARP).

Checking Log File

```
tcpdump -r /var/log/snort/snort.log.(number)
```

-r tells tcpdump to read from a saved binary log file rather than a live network interface. Replace (number) with the actual Unix timestamp suffix appended to your log file (e.g., snort.log.1779170771).



• reading from file /var/log/snort/...: Snort is analyzing a previously saved log file (snort.log.1779170771) rather than a real-time network card; the numbers at the end represent the Unix timestamp when the file was created.

• link-type EN10MB (Ethernet): The Layer 2 type is Ethernet specification. Although EN10MB historically refers to 10Mbps Ethernet, it is still displayed for modern environments.

• snapshot length 1514: The packet capture was set to truncate and save packets up to 1514 bytes. Since the standard Ethernet MTU is 1500 bytes (+ 14 bytes Ethernet header), the entire packet was captured fully.

• 06:06:37.390429: Timestamp of the captured packet.

• ARP: The Layer 3 protocol is ARP for address resolution, used when the IP address is known but the physical address (MAC) is unknown.

• Request: A "Request" packet that sends a query to the destination.

• who-has: A request asking "Who has this?".

• 2af7b0a26c0e: This hexadecimal string represents the Docker container ID assigned to the target.

• tell 172.17.0.1: Docker Bridge Interface virtual gateway address — the IP of the entity that sent this query.

• length 28: The pure size of the ARP request data is 28 bytes.

Creating Custom Rules

Creating Rules File

```
# ./snort_rules/snort.rules

alert tcp any any -> any any (content:"www.seoultech.ac.kr"; msg:"SEOULTECH is opend";
sid:123123;)
```

This is a Snort rule written into ./snort_rules/snort.rules. It alerts on any TCP packet whose payload contains the string "www.seoultech.ac.kr". msg sets the alert label shown in the console, and sid is a unique rule ID required by Snort.

Modifying snort.conf

```
# In Section #7, add next line

include $RULE_PATH/snort.rules
```

Adding this line in Section 7 of snort.conf tells Snort to load the custom rules file at startup. $RULE_PATH is a variable already defined earlier in snort.conf that points to the rules directory.

Running Snort with snort.conf

```
# Using Alert Mode Console

# So, it is not saved

snort -c /etc/snort/snort.conf -A console -i eth0
```

-c loads the specified configuration file. -A console prints alerts directly to the terminal in real time instead of writing them to a file. -i eth0 specifies the network interface to listen on.

Docker Checksum Problem

In Docker, for performance optimization, the Linux kernel leaves the packet error verification checksum blank without calculating it, sending out packets with incorrect values and effectively leaving the responsibility to the final physical network card.

However, IDS engines like Snort, upon receiving such packets with incorrect checksums, determine them to be lost or tampered forged packets; they do not even enter the rule checking (payload matching) stage and simply drop (ignore) them.

In other words, the packet passes through eth0, but Snort filters it out from behind an internal veil (checksum).

Therefore, checksum checking should be disabled in a Docker environment.

```
# TCP Checksum Offloading issue unique to Docker virtual network environments
snort -c /etc/snort/snort.conf -A console -i eth0 -k none
```

-k none disables all checksum validation in Snort. This is necessary inside Docker because the kernel offloads checksum calculation to the physical NIC, leaving packets with invalid checksums that Snort would otherwise silently drop before any rule matching occurs.

Alternatively, you can resolve this problem at the Linux kernel level:

```
# ethtool install
apt-get install -y ethtool

# Turn off the TX Check Sum Offload of eth0 interface
ethtool -K eth0 tx off
```

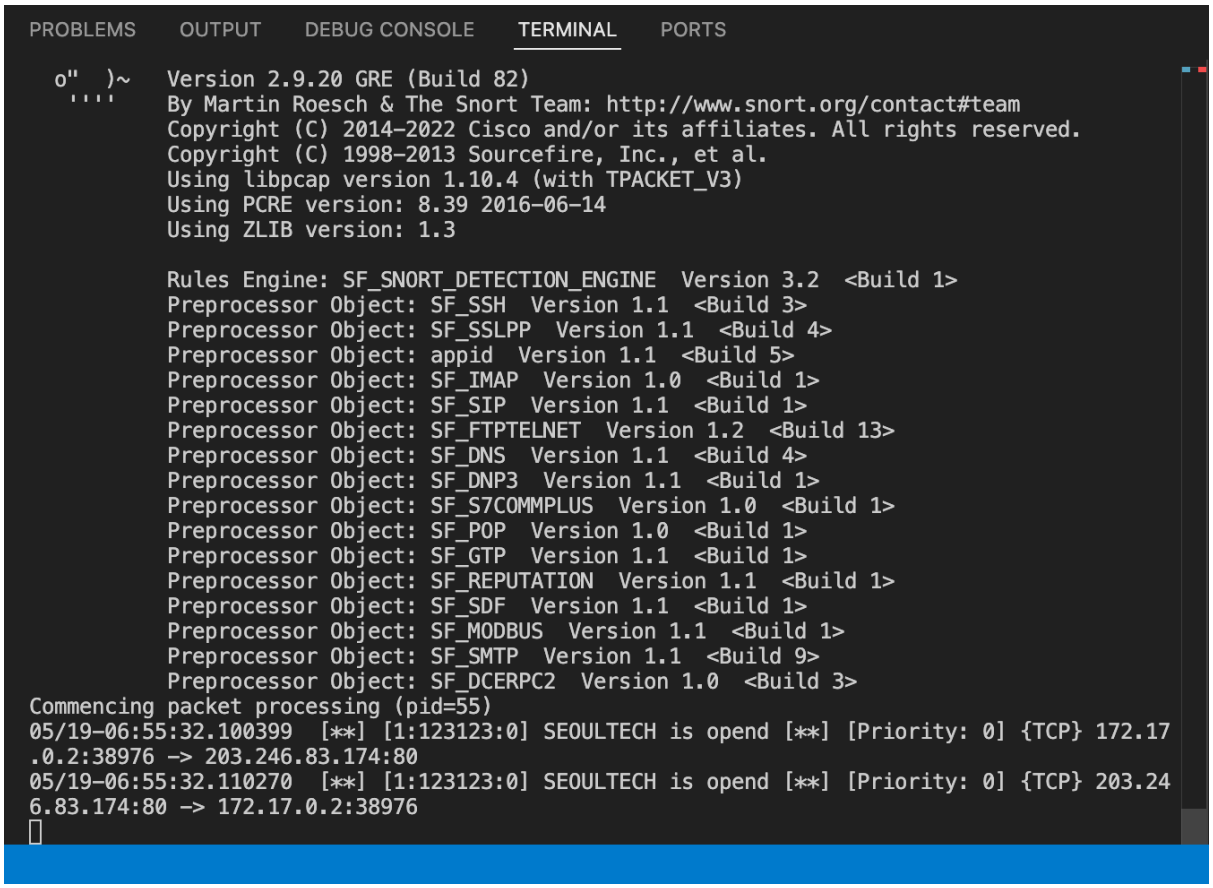
apt-get install -y ethtool installs the ethtool utility. ethtool -K eth0 tx off disables TX (transmit) checksum offloading on the eth0 interface at the kernel level, so packets are sent with correctly computed checksums that Snort can validate normally.

Execute Instruction by Another Terminal

```
curl -v http://www.seoultech.ac.kr
```

-v enables verbose output, showing the full HTTP request and response headers. This request generates TCP traffic containing "www.seoultech.ac.kr" in the payload, which triggers the custom

Snort rule defined earlier.



Creating Other Local Rules

Creating local.rules

```
# You need to replace server_ip your own server ip

# Plz check the `ifconfig` or `ipconfig`

alert tcp any any -> $(SERVER_IP) 1234 (msg:"Scanning_tmp1"; flow:stateless;
classtype:attempted-recon; sid:13;)
```

This rule alerts on any TCP packet destined for port 1234 on the server host. Replace $(SERVER_IP) with the actual IP address of the target machine (find it with ifconfig or ipconfig). flow:stateless matches regardless of connection state, making it suitable for detecting SYN scans. classtype:attempted-recon categorizes the alert as a reconnaissance attempt.

Modifying snort.conf

```
# include $RULE_PATH/snort.rules

include $RULE_PATH/local.rules
```

The previous snort.rules line is commented out and replaced with local.rules. This switches the active rule set so Snort loads only the local scanning-detection rule for this exercise.

Running Snort

```
# For WSL
snort -c /etc/snort/snort.conf -A console -i eth0

# For Docker Container on Host
snort -c /etc/snort/snort.conf -A console -i eth0 -k none
```

Use the WSL command if running Snort directly inside WSL (checksums are valid). Use the Docker command (with -k none) if running inside a Docker container, where checksum offloading causes Snort to silently drop packets otherwise.

Open Port

```
# For Docker, this line is not necessary
nc -l -p 1234
```

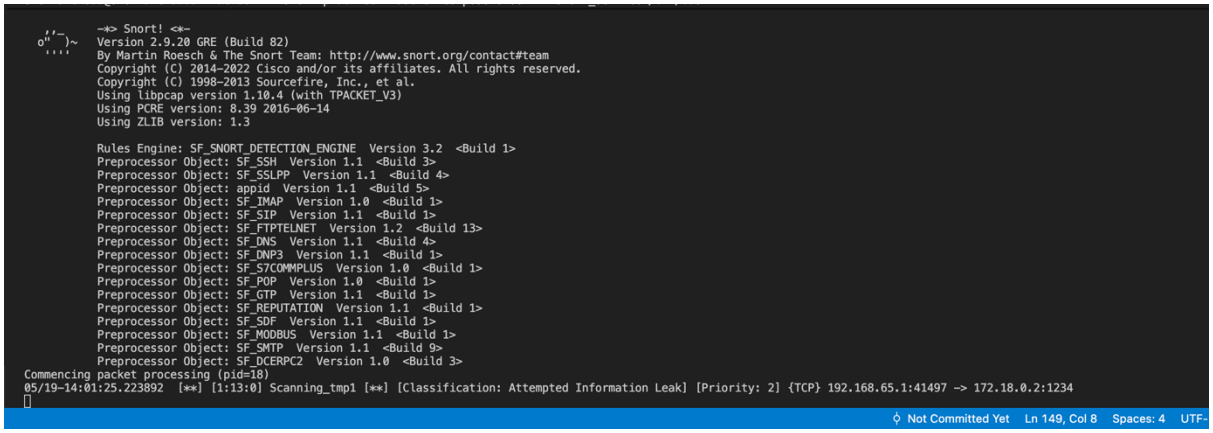
nc -l -p 1234 starts a simple TCP listener on port 1234 using Netcat, giving the scanner a reachable port to probe. In a Docker environment this step is unnecessary because the port is already exposed by the container configuration.

Scanning Port using NMAP

```
# WSL
nmap -sS -p 1234 $(SERVER_IP)

# Docker Container
nmap -sT -p 1234 $(SERVER_IP)
```

-sS performs a SYN (half-open) scan, which requires raw socket privileges and is used in WSL where that is available. -sT performs a full TCP connect scan, used inside Docker where raw sockets are typically unavailable. Both scan port 1234 on $(SERVER_IP) to trigger the Snort alert.



Creating DoS Rules

Creating dos.rules

```
alert tcp any any -> any any (msg: "DOS ATTACK IS DETECTED"; flags:S; threshold: type threshold, track by_dst, count 20, seconds 60; sid: 5000;)
```

flags:S matches only TCP SYN packets (connection-initiation packets). The threshold keyword suppresses repeated alerts and instead fires once when the count reaches 20 SYN packets within 60 seconds to the same destination (track by_dst), which is a classic sign of a SYN-flood DoS attack.

Modifying snort.conf

```
# In Section #7, append the DoS rules file
include $RULE_PATH/dos.rules
```

This switches the active rule set so Snort loads the Denial of Service detection rules for this exercise.

Running Snort

```
snort -c /etc/snort/snort.conf -A console -i eth0 -k none
```

Runs Snort using the local snort.conf in the current directory, printing alerts to the console. -k none disables checksum validation for the Docker environment.

Send TCP Packet all at once

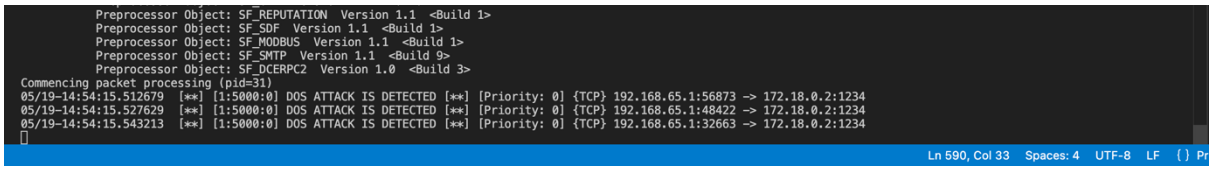
```
# For Windows
nping --tcp --flags SYN -p 80 --rate 5 --count 20 (SERVER_IP)

# For Mac
seq 60 | xargs -I {} -P 60 nc -zv -G 1 (SERVER_IP) 1234
```

Windows: nping sends 20 raw TCP SYN packets to port 80 at a rate of 5 packets per second, simulating a SYN flood.

Mac: seq 60 generates 60 numbers piped into xargs. -I {} acts as a loop controller, ensuring the command runs exactly 60 times. -P 60 spawns up to 60 parallel nc processes simultaneously. Each nc -zv initiates a TCP connection scan (transmits a raw TCP SYN packet) with a 1-second timeout (-G 1). This floods the target with concurrent SYN packets, triggering the Snort threshold.

Results



Helpful Instructions

- pkill -9 snort or kill -9 %1: Force stopping snort
- If you want to revise this project, you can access the snort_rules, also snort.conf too